

# Opis algorytmu MNK

## Spis treści:

1. Krótki wstęp
2. Jak samodzielnie testować dane
3. Opis działania algorytmu (głównie macierzy)

## Krótki opis

Istnieją aktualnie dwa algorytmu MNK v1 oraz v1.1. Oba są bardzo podobne, jedyna zmiana jest w pozyskiwaniu  $T_s$ , raz jest stałe a raz troszkę bardziej dynamiczne. Oba algorytmu przyjmują i oddają dane w ten sam sposób. Dane wejściowe są okrojone z nagłówka (można to łatwo zmienić dodając `fgets`). Dane wyjściowe to parametry  $T$ ,  $K$ ,  $T_s$  oraz równanie krzywej. Jeżeli dane są niezgodne to wyrzuca błąd i krótki opis. Pliki odpowiadają odpowiednio starym i nowym danym. Te z dopiskiem `filtr` posiadają tylko wyniki gdzie  $u[k] > 0$ .

## Testowanie danych

Pliki wejściowe składają się z mniejszych sekcji danych oddzielonych od siebie znakiem `"-,` (może być więcej niż jeden myślnik). Dane jakie muszą się znajdować w takiej sekcji to: wyjście i wejście (w naszym przypadku `Obw.` i `temp.`). Żeby przypasować kod do pliku trzeba zmienić tylko 3 rzeczy. Pierwsze dwie znajdują się na samej górze kodów, czyli `#define COL_INPUT` oraz `#define COL_OUTPUT`, trzeba w nich podać indeksy kolumn z danymi wejściowymi i wyjściowymi. Druga sprawa to nazwa pliku z jakiego korzystamy 118/174 linijka, ja używam `txt` i wiem że działa. Co do wersji v1.1 to działa on jeszcze na dwóch kolumnach do których indeksy również musimy znać `#define COL_DATE` `#define COL_TIME`.

## Opis działania algorytmu

Algorytm dzieli na sekcje dane które otrzymał i każdą rozwiązuje osobno. Pierwszym etapem jest przygotowanie otrzymanych danych pod macierz:

$$\begin{bmatrix} \sum y_{k-1}^2 & \sum y_{k-1}u_{k-1} \\ \sum y_{k-1}u_{k-1} & \sum u_{k-1}^2 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum y_k y_{k-1} \\ \sum y_k u_{k-1} \end{bmatrix}$$

Algorytm tworzy poszczególne sumy z aktualnej sekcji danych. Tutaj może się rodzić pytanie czym jest a i b. Najlepiej jest to przedstawić na obrazkach

$$y[k] = a \cdot y[k-1] + b \cdot u[k-1]$$

$$y(k+1) = y(k)e^{-\frac{T_s}{\tau}} + K(1 - e^{-\frac{T_s}{\tau}})u(k)$$

Po analizie oby tych zdjęć łatwo możemy zauważyć czym stały się parametry a oraz b. Pytanie które może się nasunąć to po co nam ta zmiana i czy ma to wpływ na wynik. Zaczynając od drugiego pytania, z tego co wiem nie ma to wpływu na wynik, co do parametru a jestem tego pewna bo e oraz  $T_s$  to są wartości stałe więc nie mają ważnego wpływu, natomiast przy b miałabym swoje wątpliwości. Natomiast wracając do punktu pierwszego ta zmiana jest dla mnie potrzebna, ponieważ na ten moment nie znalazłam innej opcji implementacji algorytmu oraz nie znalazłam żadnego dowodu że ta zmiana negatywnie wpływa na wyjściową funkcję.

Następnym krokiem jest przekształcenie tego a i b na wartości T i K. I tutaj się pojawia nam pierwszy problem wynikający z ograniczenia a. Ponieważ a musi być większe od zera, żeby obliczyć T, oraz gdy  $a > 1$  to T staje się mniejsze od zera. Ograniczenia a są dość srogie co nam mocno limituje wyniku.

Różnica pomiędzy plikami jest jedna w pliku v1 trzeba samemu z palca wybrać  $T_s$  (w kodzie). A w pliku v1.1  $T_s$  się liczy automatycznie jako średnia odległości czasów, na to liczę nie jestem świadoma co się tam dokładnie dzieje ale temu ufam.